



Container Orchestration Storage

Persistent Volumes

 Copy page

This article provides a comprehensive guide on persistent volumes in Kubernetes, focusing on centralized storage management for scalable environments.

Welcome to this comprehensive guide on persistent volumes in Kubernetes. In previous lessons, we introduced the concept of volumes. Now, we focus on persistent volumes and explore how they centralize storage management for a scalable, production-ready environment.

Traditional Volume Configuration

Previously, storage details were embedded directly into the pod specification file. For example:

```
volumes:  
- name: data-volume  
  awsElasticBlockStore:  
    volumeID: <volume-id>  
    fsType: ext4
```

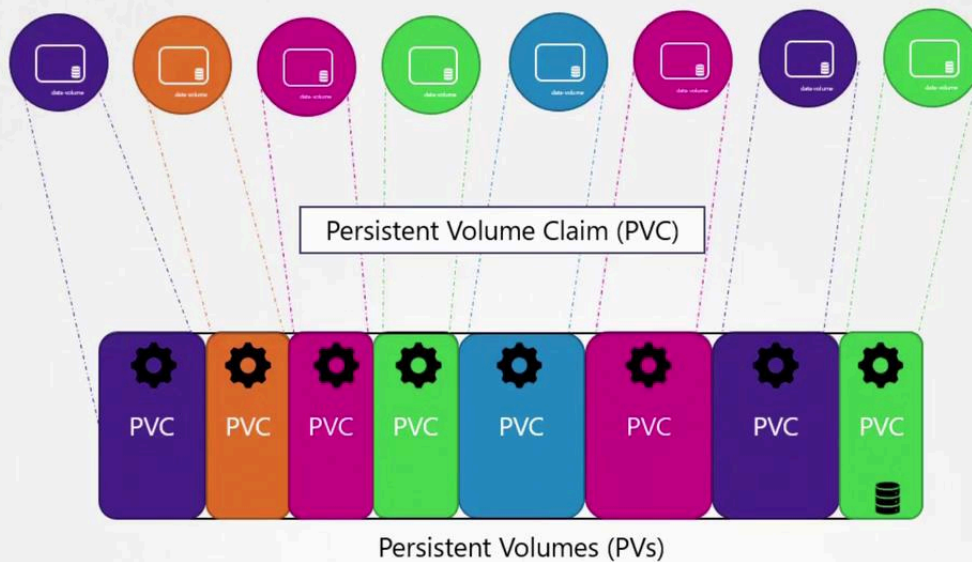


Although this approach works for simple deployments, it becomes cumbersome in large, multi-user environments. Each pod requires its own storage configuration, making global updates difficult and error-prone.

Centralized Storage with Persistent Volumes

>

Persistent Volume



In summary:

Administrators manage storage centrally.

Users claim storage as needed without repetitive configuration.

Global changes and updates become easier to implement.

Creating a Persistent Volume

In this section, we create a persistent volume using a YAML template. Begin by updating the API version, setting the kind to PersistentVolume, and naming the volume (e.g., PV-01).

Within the spec section, include:

Access Modes: Determines how a volume can be mounted (e.g., ReadOnlyMany, ReadWriteOnce, ReadWriteMany).



>

while using a `hostPath` is useful for demonstration purposes, it is not recommended for production environments. Always opt for a supported production-ready storage solution.

Below is an example of a persistent volume definition:

```
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-vol1
spec:
  accessModes:
    - ReadWriteOnce
  capacity:
    storage: 1Gi
  hostPath:
    path: /tmp/data
```



Deploying the Persistent Volume

To create the persistent volume, run the following command in your terminal:

```
kubectl create -f pv-definition.yaml
```



After deployment, verify that the persistent volume has been successfully created by executing:

```
kubectl get persistentvolume
```



This command lists all available persistent volumes, including the one you just created.



>

PVCs, you ensure efficient resource utilization and easier scalability.

For further information, visit the [Kubernetes Documentation](#).

Happy deploying!



Watch Video

[Learn more >](#)

Was this page helpful?



Yes



No

Volumes

[< Previous](#)

Persistent Volume Claims

[Next >](#)

Powered by [mintlify](#)